# Claude vs Kimi

Two AI-assisted security reviews of the same GitHub SOC laboratory | Informal public comparison | 10 October 2026

Experiment: Two AI assistants were asked to assess an open-source AI-powered SOC proof of concept integrating Wazuh, Python, Qwen, LM Studio, and VMware. This report compares the reviews actually returned, not the general capabilities of either model.

Repository: https://github.com/vitaliibalint39-oss/ai-powered-soc-lab

Kimi review commit: 9be444ff7c271711d90920dbdac2bf62e3bbf382

## Executive takeaway

Claude produced a transparent, useful README-based architecture review after reporting failures to access code. Kimi reported a broader static review of source, documents, dependencies and screenshots, giving specific file/line references and additional findings. The differing scope of material accessible to each assistant is a major confounding factor; this is not a controlled benchmark.

## Side-by-side methodology

| Dimension | Claude | Kimi |
|---|---|---|
| Access claimed | Landing page + README only; source access failed (503) | Static source, documents, dependencies, examples, screenshots |
| Live deployment tested? | No | No |
| Line-level references | No; checks explicitly marked unverified | Yes; source links and line ranges provided |
| Main emphasis | Risk identification and review checklist | Specific implementation gaps and remediation roadmap |
| Limitations disclosed | Source not reviewed | Static review; no live Wazuh/LM Studio execution |

Evidence comes from the two supplied audit documents; neither report represents a deployed security certification.

# What the models found

## Overlapping findings

• Potential alert loss from fetching only the latest 20 matching Wazuh alerts.

• Prompt-injection risk when attacker-controlled event text reaches the local LLM.

• Risk around non-transactional report/state handling and duplicate or incomplete processing.

• Lack of automated testing, CI, and reproducible dependency management.

• Need for model-output quality evaluation, safe reporting, and explicit analyst review.

## Additional specificity from Kimi

• Identified a screenshot showing an LM Studio endpoint on a non-loopback IP; actual network exposure requires verification on the host.

• Reported silent fallback to the first available model if the expected Qwen model is missing.

• Pointed to acceptance of non-empty free-form LLM output without schema validation.

• Called for reproducible least-privilege OpenSearch role definitions and negative authorization tests.

• Flagged screenshots with operating-system usernames, paths, internal network details, and inconsistent older example output.

## Differences in interpretation

Not “bugs found” vs “bugs missed.” Claude could not inspect the source and framed many points as hypotheses/checks. Kimi describes code-level findings. Both can be useful, but only findings confirmed against the referenced commit should be published as verified defects.

Potential security exposure is not proof of internet exposure. A non-loopback listener may be restricted to a private, virtual, or overlay network. Local listener checks and firewall review are necessary.

Precision matters more than volume. A larger report, more tool calls, or more proposed phases does not alone establish a more accurate assistant.

## Publication status

The code-level points are candidates for direct verification against the pinned commit; LM Studio reachability and overall deployed security were not tested.

# Conclusions & reproducibility

## What this experiment shows

In this single run, Claude emphasized transparency about failed source retrieval and architectural risk assessment. Kimi produced a more detailed static-review artifact with concrete references. Different information access prevents a fair quality ranking. Both outputs helped prioritize engineering work.

## Suggested next checks

• Confirm findings directly against the pinned Git commit, with minimal reproduction steps and test outputs.

• Check LM Studio bind addresses, firewall rules, and access from another machine or network segment.

• Implement pagination and state-management tests using at least 1,000 synthetic events.

• Review screenshots and reports for sensitive information before publishing them or linking to the repo.

• Rerun both models with identical, accessible source snapshots and a predefined scoring rubric if a more controlled comparison is desired.

## Sources and links

Claude: Security & Architecture Review: ai-powered-soc-lab, 5-page PDF supplied by project owner; explicitly README-only.

Kimi: Repository security and AI architecture review, supplied response and formatted PDF; pinned review of commit 9be444ff7c271711d90920dbdac2bf62e3bbf382.

Source repository: https://github.com/vitaliibalint39-oss/ai-powered-soc-lab

Pinned code: soc_assistant.py at audited commit

## Publication note

This document is an independent comparison prepared from the two supplied AI reports. It does not assert an independent line-by-line re-audit, execution of the laboratory, or an official affiliation with Anthropic or Moonshot AI. Keep the original PDFs together with this comparison so readers can inspect the source material.

---

Prepared for educational discussion and portfolio documentation. No winner is declared.